

Comparaison fonctionnelle — Monolithe (LabScheduling) vs Clean Architecture (lab_scheduler)

Analyse fonction par fonction, fondée sur le code source réel des deux versions.
Objectif : déterminer précisément quels éléments de la version Clean Architecture méritent d'être portés dans le monolithe, et lesquels y existent déjà.

Périmètre analysé :

- **Monolithe** : `pipeline.py`, `professor_credits.py`, `lab_professor_assignment.py`, `validation_credits.py`, `excel_generator_core.py`, `excel_export.py`, `reliability_metrics.py`, `kpi_report.py`, `tests/`.
 - **Clean Architecture** : `domain/rules.py`, `domain/entities/*`, `application/services/*`, `infrastructure/solver/*`, `infrastructure/excel/*`, `infrastructure/config/config_loader.py`, `tests/`.
-

1. Résumé exécutif

Conclusion principale : le monolithe couvre déjà l'essentiel des fonctionnalités métier de la version Clean Architecture. La règle « 1 crédit P = 5 séances », l'ingestion de l' `Asignacion_2025-2026_v5.xlsx`, le découpage T/P, la validation des crédits, l'assignation **fixe d'un professeur par groupe** et la « Teacher View » consolidée **sont toutes présentes dans le monolithe.**

L'idée — formulée lors d'analyses antérieures — selon laquelle le delta majeur serait « assignation fixe professeur-session (Clean Arch) vs rotation (+N) (monolithe) » est **partiellement obsolète** : le monolithe a depuis ajouté `lab_professor_assignment.py` et la feuille « **Teacher View** » qui **assignent bien un professeur fixe par groupe** au prorata des crédits P (`excel_generator_core.py`, ligne 1764 : `_lpa.assign_schedule_groups(...)`).

Les écarts réels qui subsistent sont, par ordre d'importance :

#	Écart	Type	Priorité
1	L'assignation prof→groupe n'est pas un attribut de données du planning (calculée seulement à l'écriture Excel, jamais persistée dans le CSV maître)	Modèle de données	P0
2	L'ancienne feuille « Vista profesor » (grille) affiche toujours la rotation (+N) , en contradiction visuelle avec « Teacher View »	Cohérence UX	P1
3	Aucun test unitaire ne couvre lab_professor_assignment.py, professor_credits.py, validation_credits.py (logique métier critique non testée)	Qualité / non-régression	P1
4	La « Teacher View » liste les créneaux agrégés (« — N sessions »), pas chaque séance individuelle comme la Clean Arch	Granularité rapport	P2
5	Configuration hard-codée (LAB_CONFIG dans pipeline.py) vs config.yaml externalisé	Maintenabilité	P2
6	Absences d'infrastructure « production » : logging structuré, gestion d'erreurs typée, rate-limiting, page santé,	Robustesse opérationnelle	P2

#	Écart	Type	Priorité
	cache (présentes en Clean Arch)		

Recommandation de cap : ne PAS réécrire le monolithe. Porter, de façon ciblée, (a) la persistance de l'assignation prof→séance dans le planning, (b) l'unification de l'affichage professeur, et (c) une batterie de tests sur la logique crédits/assignation. Le reste (config YAML, infra prod) est optionnel et à arbitrer selon le besoin de déploiement.

2. Tableau comparatif détaillé par fonctionnalité

2.1 Système de crédits (« 1 crédit P = 5 séances »)

Aspect	Monolithe	Clean Architecture	Verdict
Constante de conversion	<code>CREDIT_TO_SESSIONS = 5</code> répété dans <code>professor_credits.py</code> , <code>lab_professor_assignment.py</code> , <code>validation_credits.py</code> , <code>excel_generator_core.py</code>	DE- <code>FAULT_SESSIONS_PER_CREDIT = 5</code> centralisé dans <code>domain/rules.py</code> , surchargé par <code>config.yaml</code> (<code>credit_system.sessions_per_credit</code>)	Équivalent fonctionnellement , mais le monolithe duplique la constante (risque de dérive)
Conversion crédits→séances	Inline : <code>credits * CREDIT_TO_SESSIONS</code>	Fonction pure testée <code>credits_to_sessions(credits, sessions_per_credit)</code> avec validation (<code>ValueError</code> si négatif / facteur 0)	Clean Arch plus robuste et testée
Calcul charge labo par prof	<code>professor_credits.professor_lab_load()</code> — pivot T/P, jointure budget, <code>lab_sessions = lab_credits * 5</code>	<code>CreditSystem.expected_by_professor()</code> + <code>Professor.expected_sessions()</code>	Équivalent ; le monolithe ajoute en plus la comparaison au budget (<code>Asignación recomendada</code>) absente de Clean Arch
Découpage TP (théorie+pratique)	<code>effective_p_credits()</code> — répartit le résidu P entre blocs TP au prorata (<code>lab_professor_assignment.py</code>)	<code>ExcelReader.read_assignments()</code> — distribution proportionnelle de la part P selon le ratio T/P	Équivalent

Citation — monolithe (`lab_professor_assignment.py`) :

```
CREDIT_TO_SESSIONS = 5
SESSIONS_PER_GROUP = 5 # chaque groupe de pratiques = 5 séances
```

Citation — Clean Arch (`domain/rules.py`) :

```

DEFAULT_SESSIONS_PER_CREDIT = 5
def credits_to_sessions(practice_credits, sessions_per_credit=DE-
FAULT_SESSIONS_PER_CREDIT):
    if practice_credits < 0: raise ValueError(...)
    if sessions_per_credit <= 0: raise ValueError(...)
    return int(round(practice_credits * sessions_per_credit))

```

2.2 Assignment professeur → séance

Aspect	Monolithe	Clean Architecture	Verdict
Modèle d'assignation	Fixe par groupe , au prorata des crédits P : build_group_professor_map() / assign_schedule_groups() (plus fort reste)	Fixe par groupe/séance : SchedulerService.allocate_professors() puis propagé à LabSession.professor	Logique équivalente (les deux respectent crédits P x 5)
Où vit l'assignation	Calculée à la volée lors de l'écriture Excel (build_vue_professeur (champ explicite de master_schedule.csv non persistée dans master_schedule.csv	Attribut de domaine LabSession.professor (champ explicite de l'entité, présent dans tout le pipeline)	ÉCART P0 — le monolithe ne stocke pas le prof par séance
Affichage « par séance »	Feuille « Teacher View » : 1 ligne par (prof, matière), horaire agréé par créneau récurrent	Feuille « Teacher view » : 1 ligne par (prof, matière) + chaque séance détaillée (G1 Session 1: ...)	Clean Arch plus granulaire
Affichage « grille »	Feuille « Vista professeur » : grille horaire avec rotation Prof. : Nom (+N)	Feuille « Subject view » : 1 prof unique par séance, sans +N	ÉCART P1 — incohérence interne au monolithe
Repli théorie (matière sans crédit P)	theory_professors() — affiche le prof de théorie au lieu de « N/A »	Non géré explicitement	Monolithe supérieur sur ce point
Robustesse sans le xlsx source	Cache JSON committé lab_professor_weight_s.json (correction « N/D »)	Dépend de la lecture directe du fichier	Monolithe supérieur (déploiement sans données)

Citation — monolithe (excel_generator_core.py , l.1764) : assignation fixe déjà en place

```
sgmap = _lpa.assign_schedule_groups(_fp, subject_to_groups)    # {(matière, groupe):
prof}
_exp = _lpa.expected_sessions(_fp)
```

Citation — monolithe (excel_generator_core.py , l.646-658) : rotation encore présente dans « Vista profesor »

```
# Pick ONE eligible professor to show for this cell, ROTATING across ...
return f"{names[i]} ({len(names) - 1})"
```

Citation — Clean Arch (domain/entities/group.py) : prof = donnée de première classe

```
@dataclass
class LabSession:
    ...
    professor: Optional[str] = None    # un prof responsable par séance
```

2.3 Solveur / contraintes

Contrainte	Monolithe (pipeline.py)	Clean Arch (solv- er_engine.py / con- straint_manager.py)	Verdict
Variable de décision	<code>week_vars[s['id']]</code> (IntVar par séance)	<code>ConstraintManager.create_week_variables()</code> (IntVar par séance)	Identique
C1 — pas 2× même matière/créneau	Oui (I.3532)	<code>add_no_same_subject_slot()</code>	Identique
C4 — pas 2× même salle/créneau	Oui (I.3547)	<code>add_no_same_room_slot()</code>	Identique
C5 — ordre chronologique	Oui (I.3579 <code>week_vars[k+1] > week_vars[k]</code>)	<code>add_chronological_order()</code>	Identique
Réservations (soft)	Oui (I.3563 <code>resv_</code>)	<code>add_reservation_penalties()</code>	Identique
Parité de semaine (soft)	Oui (I.3611 <code>parity_bit</code>)	<code>add_parity_penalties()</code>	Identique
Pénalité vendredi (soft)	<code>fri-day_placement_penalty()</code> (I.398)	<code>fri-day_placement_penalty()</code> (domain/ rules.py)	Identique
Objectif (ancrage + espacement)	Oui (I.3687 <code>objective_terms</code>)	<code>_build_objective()</code>	Identique
Config solveur reproductible	<code>configure_solver()</code> (seed, gap, workers, time limit)	<code>CPSATSolver._configure()</code>	Identique
Découpage par semestre	Oui	<code>solve()</code> regroupe par semestre	Identique

Verdict global solveur : strictement équivalent. Le moteur CP-SAT du monolithe est plus riche (gestion fine des groupes partagés, programmes, diagnostics d'infaisabilité `diagnose_infeasibility`, warm-start `add_week_hints`). **Rien à porter ici.**

2.4 Génération Excel

Feuille / capacité	Monolithe	Clean Architecture	Verdict
Composition des groupes	<code>build_grupos_sheet()</code>	<code>_sheet_groups()</code>	Équivalent
Vue par matière (grille horaire)	<code>build_vista_profesor_sheet()</code> (grille riche par programme, légende, fériés)	<code>_sheet_subject_view()</code> (1 ligne / séance)	Monolithe plus riche visuellement, mais incohérent sur le prof (cf. 2.2)
Vue par professeur	<code>build_vue_professeur_consolidada_sheet()</code> → « Teacher View »	<code>_sheet_teacher_view()</code>	Équivalent , granularité moindre côté monolithe
Feuille de validation crédits	Fichier séparé <code>validation_credits_professeurs.xlsx</code> (<code>validation_credits.py</code>)	Onglet <code>_sheet_validation()</code> intégré au classeur	Différence de packaging seulement
Mise en forme (bandes, couleurs OK/Gap)	Oui (très soignée : fériés, programmes, calendrier)	Oui (plus sobre)	Monolithe plus abouti

Verdict : le monolithe produit un Excel plus complet et plus présentable. Seule la granularité « une ligne par séance » de la Teacher View Clean Arch est un plus à porter (P2).

2.5 Validations & détection de conflits

Mécanisme	Monolithe	Clean Architecture	Verdict
Validation crédits attendus vs planifiés	<code>validation_credits.build_report()</code> — rapport 3 feuilles (résumé, détail prof, méthodologie)	<code>AssignmentReporter.report_lines()</code> / <code>CreditValidator.validate()</code>	Équivalent (monolithe plus détaillé en sortie)
Détection de conflits post-solve	<code>reliability_metrics.detect_conflicts()</code> — C1, C4, conflits étudiants	<code>ConflictDetector.detect()</code> — C1, C4, C5 , double-booking étudiant	ÉCART mineur : le monolithe ne re-vérifie pas C5 en post-solve
Tolérance paramétrable	Non explicite	<code>CreditValidator(tolerance=...)</code>	Clean Arch plus souple
Métriques de fiabilité	Très riche : couverture, distribution, occupation salles, surcharge étudiants, qualité d'espacement, health score 0-100 (<code>reliability_metrics.py</code> , 918 l.)	Limité	Monolithe nettement supérieur

Citation — monolithe (`reliability_metrics.py`, l.548) :

```
# Defensive check: even though the solver guarantees C1/C4, we re-verify.
```

Note : le commentaire ne mentionne que C1/C4 ; **C5 n'est pas re-vérifié** côté monolithe, alors que `ConflictDetector` Clean Arch le fait. Petit filet de sécurité à ajouter (P2).

2.6 Couverture de tests

Domaine testé	Monolithe (tests/)	Clean Arch (tests/)
Constantes / config solveur	✓ test_solver_config.py , test_problem_constants.py	✓ test_config_loader.py
Warm-start / diagnostics in-faisabilité	✓ test_solver_config.py	—
Qualité des données	✓ test_data_quality.py	—
KPI	✓ test_kpi_report.py	—
Règle crédits → séances	✗ absent	✓ test_credit_system.py (paramétré, cas négatifs)
Validation surcharge/sous-charge	✗ absent	✓ test_credit_system.py , test_assignment_report.py
Assignment prof par crédits	✗ absent	✓ test_solver_engine.py::test_sched- uler_service_allocates_prof essors_by_credits
Détecteur de conflits (C1/ C4/C5/étudiant)	⚠ indirect	✓ test_conflict_detector.py
Génération Excel (sheets, 1 prof/séance)	✗ absent	✓ test_assignment_report.py
Lecture fichier réel	✗	✓ test_integration_real_file. py (skip si absent)
Solveur end-to-end	⚠ partiel	✓ test_solver_engine.py

Verdict : la logique métier critique du monolithe (crédits, assignment, validation)

n'est PAS testée unitairement. C'est le risque de non-régression le plus important maintenant que cette logique vit dans `lab_professor_assignment.py` / `professor_credits.py` .

2.7 Configuration & infrastructure

Aspect	Monolithe	Clean Architecture	Verdict
Config matières/con- traintes	LAB_CONFIG hard- codé dans pipeline.py + sur- charges user_config.json (ap- ply_user_config())	config.yaml extern- alisé, typé via Confi- gLoader / AppSettings	Clean Arch plus maintenable ; monolithe fonctionnel mais rigide
Logging structuré	print() dispersés	infrastructure/log- ging_config.py	Clean Arch supérieur
Gestion d'erreurs typée	Try/except + warn- ings	infrastructure/er- rors.py (AppError , SolverError , handle_errors)	Clean Arch supérieur
Rate limiting / santé	—	infrastructure/se- curity.py , infra- structure/health.py	Spécifique Clean Arch
Cache	Cache JSON poids profs (committé)	infrastructure/ cache.py	Approches différentes
Déploiement	Build PyInstaller / in- stalleur Windows (installer.iss , build.bat)	Docker / docker-com- pose / systemd	Cibles différentes (desktop vs serveur)

3. Liste priorisée des éléments à porter dans le monolithe

P0 — Impact métier élevé, à faire

P0.1 — Persister l'assignation professeur par séance dans le planning

- **Description** : aujourd'hui l'assignation prof→groupe est recalculée uniquement à l'écriture de la feuille « Teacher View ». Elle n'est **pas écrite** dans master_schedule.csv / optimized_schedule_v5.csv . Conséquence : impossible de consommer cette donnée ailleurs (KPI, validation, ré-édition manuelle), et tout consommateur doit ré-importer le xlsx source.
- **Cible (Clean Arch)** : LabSession.professor est un champ persistant du modèle.
- **Fichiers concernés** : pipeline.py (écriture du schedule_df / CSV de sortie), lab_professor_assignment.assign_schedule_groups() (déjà existant — à appeler en amont), excel_generator_core.py (consommer la colonne au lieu de recalculer).

- **Effort** : **M** ($\frac{1}{2}$ -1 j). Ajouter une colonne `professor` au DataFrame de planning après le solve, alimentée par `assign_schedule_groups()`, puis l'utiliser partout.
- **Dépendances** : aucune (la logique d'allocation existe déjà).

P1 — Cohérence & non-régression, fortement recommandé

P1.1 — Unifier l'affichage professeur (supprimer la rotation (+N))

- **Description** : la feuille « Vista profesor » montre `Prof.: Nom (+N)` (rotation), tandis que « Teacher View » montre l'assignation fixe. C'est incohérent pour l'utilisateur final.
- **Cible (Clean Arch)** : « Subject view » affiche **un seul prof** par séance.
- **Fichiers concernés** : `excel_generator_core.py` (`format_lab_session_label` l.602-658, `build_vista_profesor_sheet` l.1255). Utiliser `sgmap` (assignation fixe) au lieu de la rotation pour résoudre le prof de chaque (matière, groupe).
- **Effort** : **M** ($\frac{1}{2}$ j). Dépend idéalement de P0.1 (colonne `professor` disponible).
- **Dépendances** : P0.1 (recommandé).

P1.2 — Ajouter des tests unitaires sur la logique crédits/assignation

- **Description** : porter les cas de test Clean Arch qui couvrent la logique métier désormais critique du monolithe.
- **Cas à porter** (depuis `test_credit_system.py`, `test_assignment_report.py`, `test_conflict_detector.py`, `test_solver_engine.py`) :
 - conversion crédits→séances (3→15, 2→10, 0→0, négatif → erreur) ;
 - `professor_lab_load` : surcharge/sous-charge, somme P, découpage TP ;
 - `effective_p_credits` / `_allocate_groups` : plus fort reste, Σ = budget ;
 - `assign_schedule_groups` : Física (5+5+3+2 → groupes 1..15), proportions respectées ;
 - `validation_credits.build_report` : écart attendu vs planifié, alertes ;
 - `detect_conflicts` : C1/C4/étudiants (+ C5, cf. P2.2).
- **Fichiers concernés** : nouveaux `tests/test_professor_credits.py`, `tests/test_lab_professor_assignment.py`, `tests/test_validation_credits.py`.
- **Effort** : **M-L** (1-1,5 j).
- **Dépendances** : aucune.

P2 — Améliorations, à arbitrer

P2.1 — Détailler chaque séance dans la « Teacher View »

- **Description** : la Clean Arch liste chaque séance (`G1 Session 1: Lundi 08:30-10:30 ...`) au lieu d'agréger par créneau récurrent (« — N sessions »).
- **Fichiers concernés** : `excel_generator_core.py` (`_format_session_timetable` l.1661).
- **Effort** : **S** (2-3 h). Ajouter un mode « détail par séance ».

P2.2 — Re-vérifier C5 (ordre chronologique) en post-solve

- **Description** : `detect_conflicts` re-vérifie C1/C4/étudiants mais pas C5.
- **Fichiers concernés** : `reliability_metrics.detect_conflicts()` (l.545).
- **Effort** : **S** (1-2 h).

P2.3 — Centraliser la constante `CREDIT_TO_SESSIONS`

- **Description** : la valeur 5 est dupliquée dans 4+ fichiers. Risque de dérive.
- **Fichiers concernés** : créer un petit module `constants.py` (ou réutiliser un module existant) importé partout.

- **Effort** : S (1 h).
- **Dépendances** : aucune, mais toucher plusieurs fichiers → bien tester (cf. P1.2).

P2.4 — (Optionnel) Externaliser `LAB_CONFIG` vers un YAML

- **Description** : améliorer la maintenabilité en sortant la config des matières du code.
- **Fichiers concernés** : `pipeline.py` (`LAB_CONFIG` , `apply_user_config`).
- **Effort** : L (2-3 j) — risqué, beaucoup de code dépend de `LAB_CONFIG` .
- **Recommandation** : **ne pas faire** sauf besoin explicite ; le `user_config.json` couvre déjà la surcharge dynamique.

P2.5 — (Optionnel) Infrastructure « production » (logging, erreurs typées, santé)

- **Description** : utile uniquement si le monolithe doit devenir un service serveur. Pour une application desktop packagée (PyInstaller), faible valeur ajoutée.
- **Recommandation** : **différer** ; ne porter que `logging_config` si un vrai besoin de traçabilité apparaît.

4. Recommandations d'implémentation

1. **Ne pas réécrire.** Le monolithe est fonctionnellement au niveau de la Clean Arch sur le métier, et **supérieur** sur le solveur, les métriques (health score) et le rendu Excel. Une migration Clean Arch n'apporterait pas de valeur métier proportionnée au risque.
2. **Séquencer le portage** :
 - **Étape 1 (P0.1)** : ajouter la colonne `professor` au planning de sortie en appelant `assign_schedule_groups()` juste après le solve. C'est le socle qui débloque P1.1 et fiabilise tout le reste.
 - **Étape 2 (P1.2)** : écrire les tests AVANT de toucher davantage à l'affichage, pour figer le comportement attendu (filet de non-régression).
 - **Étape 3 (P1.1)** : unifier l'affichage prof en s'appuyant sur la colonne `professor` .
 - **Étape 4 (P2.x)** : améliorations cosmétiques/robustesse au fil de l'eau.
3. **Garde-fous** :
 - Toute modification doit conserver la philosophie du projet :
« l'affectation est une donnée ; le système la valide, il ne la décide pas » (signaler les écarts, ne jamais bloquer).
 - Préserver le **cache JSON committé** (`lab_professor_weights.json`) : c'est ce qui permet à la Teacher View de fonctionner sans le xlsx source en déploiement.
 - Lancer la suite de tests existante + nouvelle après chaque étape ; vérifier que le smoke test « 1 crédit P = 5 séances » reste vert (ex. Física : 6 crédits → 30 séances).
4. **À NE PAS porter** : le moteur solveur, le découpage par semestre, les diagnostics d'infaisabilité, le rendu calendrier/fériés — le monolithe est déjà plus complet.

5. Synthèse en une phrase

Le monolithe **possède déjà** la quasi-totalité des fonctionnalités de la Clean Architecture (y compris l'assignation fixe d'un professeur par groupe) ; les seuls portages à réelle valeur sont **(P0)** persister le professeur dans le planning, **(P1)** unifier l'affichage et **(P1)** couvrir la logique crédits/assignation par des tests — le reste relève du confort ou d'un futur déploiement serveur.